2024 International Conference on Electrical Electronics and Computing Technologies (ICEECT) | 979-8-3503-7809-2/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICEECT61758.2024.10738945

# Artificial Intelligence in System and Software Engineering for Auto Code Generation

Anupriya Sharma Ghai
*School of Computing*
*Graphic Era Hill University*
Dehradun, India
anupriya@gehu.ac.in

Vandna Rawat
*Department of Computer Science Engineering*
*Graphic Era Deemed to be University,*
Dehradun, India
vandnarawat2405@gmail.com

Vishan Kumar Gupta
*School of Engineering and Technology*
*Amity University. Punjab*
Mohali India
vishangupta@gmail.com

Kapil Ghai
*Department of Chemistry*
*Graphic Era Hill University*
Dehradun , India
kapilghaiddn@yahoo.co.in

***Abstract*—Artificial Intelligence has profoundly impacted system and software engineering by facilitating the automation of complicated tasks, reducing errors, and accelerating the development process. AI-driven tools help enhance efficiency, improve code quality, and promote the generation of more adaptive and innovative software solutions. The present research uses AI-generated tools to explore how AI transforms code synthesis. The study will include a comparative analysis of the performance of manual coding and other automated methods to explore how AI can accelerate code synthesis and make modern web development more competent or effective. It is suggested in the research that with algorithmic and contextual consideration, the quality of code developed by the AI would be higher in most cases than by people due to lower error rates and shorter synthesis. Overall, the central conclusion of the research is that AI plays a crucial role in transforming systems and software engineering. The paper also includes information on ethical dilemmas, robust validation mechanisms, conservative modes, and current research.**

***Keywords—Artificial intelligence, system and software engineering, ai code generation, manual code.***

## I. INTRODUCTION

Artificial Intelligence (AI) has traversed a transformative journey over the decades, leaving an indelible mark on the engineering landscape. The evolution of AI is not just a chronological progression; it is a narrative of breakthroughs and paradigm shifts that have redefined the possibilities within the realm of technology and engineering. The journey of AI can be traced back to its nascent stages in the mid-20th century, where the pioneering work of visionaries like Alan Turing and John McCarthy laid the groundwork for the conceptualization of machines endowed with intelligent reasoning and problem-solving capabilities [1].

These early visionaries set the stage for a revolution that would go on to reshape the very fabric of engineering practices. The significant milestones in AI's evolution have propelled it beyond its initial theoretical constructs, ushering in practical applications that permeate various engineering domains. Machine learning algorithms, neural networks, and advanced computational models have become the bedrock of AI, enabling machines to mimic human intelligence and surpass it in specific tasks. In data processing and analysis, AI has emerged as an unparalleled force. The advent of sophisticated algorithms has empowered AI systems to sift through vast datasets, uncovering patterns, trends, and insights that would be inconceivable through traditional analytical methods [2]. This capacity for data-driven decision-making has become a cornerstone in engineering, influencing strategic planning, risk management, and innovation.

Integrating AI into engineering practices extends beyond mere automation; it represents a paradigm shift in problem-solving approaches. AI technology efficiency is instrumental in optimizing complex processes, enhancing operational efficiency, and pushing the boundaries of what was once achievable. From autonomous vehicles to smart cities, AI's influence resonates in the infrastructure that underpins modern engineering marvels.

Furthermore, the collaboration between AI and other cutting-edge technologies, such as the Internet of Things (IoT) and robotics, has given rise to cyber-physical systems that redefine engineering possibilities. The interconnectedness facilitated by AI-driven systems enables real-time adaptation, predictive maintenance, and unprecedented levels of efficiency across diverse applications. The evolution of AI stands as a testament to the unrelenting pursuit of innovation within the engineering domain. From its conceptual origins to its current state of pervasive influence, AI continues to shape the landscape in ways that were once the realm of science fiction.

The future promises even greater integration, where AI, alongside other emergent technologies, will forge new frontiers and redefine the essence of what is achievable within the vast realm of engineering. The infusion of AI into system and software engineering has brought about profound transformations, fundamentally reshaping conventional practices and methodologies. AI technologies have become integral in streamlining and automating various software development life cycle phases, resulting in heightened operational efficiency [3]. Automated processes and intelligent decision-making driven by AI accelerate

979-8-3503-7809-2/24/$31.00 ©2024 IEEE

development timelines, allowing for the swift delivery of robust software solutions [4]. The impact is not merely temporal; AI-driven testing, maintenance, and design approaches significantly improve software quality, minimize errors, and enhance overall reliability [5]. One of the critical contributions of AI in software engineering lies in its adaptability to evolving project requirements, fostering increased flexibility in engineering practices [6]. AI systems showcase a remarkable ability to learn from data patterns, dynamically adjusting their responses.

This adaptability optimizes development processes and ensures software solutions remain resilient in changing project landscapes. AI's influence extends beyond the development phase, encompassing system integration. Intelligent automation facilitated by AI ensures the seamless integration of diverse software components, promoting interoperability and mitigating integration challenges. This results in a more cohesive and efficient software ecosystem. AI-driven analytics play a pivotal role in project management and decision-making processes. The ability of AI systems to analyze vast datasets empowers project managers to make informed decisions, optimize resource allocation, and mitigate potential risks. This data-driven approach enhances overall project governance and significantly contributes to the success of software engineering endeavours [7].

Integrating AI into system and software engineering is a transformative force, revolutionizing traditional methodologies and enhancing software development life cycle efficiency, reliability, and adaptability. The continued evolution of AI is expected to shape the future landscape of engineering practices, offering innovative solutions to complex challenges in the digital era. As this research unfolds, specific case studies and real-world implementations will be explored to provide concrete examples of AI's impact on operational efficiency, development time, and software quality [8]. These examples will further emphasize the practical implications and significance of incorporating AI into engineering practices.

## II. Literature Review

The evolutionary trajectory of Artificial Intelligence (AI) in Software Engineering unfolds as a dynamic narrative, shaping the very fabric of modern computing. Tracing its roots to the mid-20th century, luminaries like Alan Turing laid the theoretical foundations that would set the stage for a transformative journey [9]. The initial exploration saw the emergence of symbolic AI and rule-based systems, serving as the rudimentary building blocks for subsequent advancements. The 1980s marked a significant turning point with the advent of expert systems. Embodying domain-specific knowledge, these systems aimed to replicate human expertise in particular domains.

The use of rule-based programming languages, exemplified by Prolog, facilitated the development of intelligent systems capable of decision-making within defined contexts. As the 21st century unfolded, the landscape of AI in Software Engineering underwent a seismic shift with the machine learning revolution. This epoch was characterized by integrating machine learning algorithms fueled by abundant large datasets. Software applications now could engage in data classification, pattern recognition, and predictive modelling, ushering in a new era of intelligent computing [10]. Deep learning has emerged as a dominant force in recent years, redefining the AI landscape. Neural networks, especially those with multiple layers, have demonstrated unparalleled success in diverse applications, including image and speech recognition, natural language processing, and complex decision-making.

This era of deep learning has refined existing applications and paved the way for novel and more sophisticated uses of AI in software engineering. Contemporary trends underscore the pervasive integration of AI throughout the software development lifecycle. AI tools now play a pivotal role in tasks such as code generation, bug detection, and software testing, contributing to increased efficiency and streamlined development processes.

This seamless integration reflects a symbiotic relationship between traditional software development practices and the transformative capabilities offered by AI. As AI's role expands, ethical considerations and the need for explanatory AI become paramount. The focus on transparency and responsible deployment aims to address potential biases and ensure that AI applications align with ethical standards. This dual emphasis on efficiency and ethical practices underscores the maturation of AI in the field of Software Engineering, promising continued innovation and positive impact in the years to come [18].

The current state-of-the-art AI techniques within the software engineering domain showcase a dynamic landscape, continually evolving to address the challenges posed by complex software development processes. This discourse will elucidate critical advancements in artificial intelligence applied to software engineering, emphasizing their implications for efficiency, productivity, and code quality. The following are the domains of AI in software engineering [19].

(i) The domain of automated code generation has witnessed a paradigm shift with the integration of state-of-the-art machine learning models. Noteworthy developments include deep learning architectures, such as OpenAI's GPT-3, which has demonstrated proficiency in understanding and generating human-like text. Researchers have explored methods to fine-tune these models on vast code repositories, enabling them to capture intricate coding patterns and semantics [11]. By doing so, these models provide developers with intelligent code suggestions and, in some instances, can autonomously generate functional code segments, expediting software development [12].

(ii) AI-powered tools have become instrumental in bug detection and resolution. Techniques such as static code analysis and dynamic program analysis, coupled with machine learning, enable these tools to predict potential issues by learning from historical data [13]. Furthermore, emerging solutions use reinforcement learning to fix identified bugs automatically, contributing to improving software robustness.

(iii) Code review processes have transformed by adopting AI-driven tools. These tools utilize machine learning algorithms to analyze code for adherence to best practices, style guidelines, and potential issues [14]. Notable advancements include integrating natural language processing (NLP) techniques for more context-aware code reviews. This ensures faster review cycles and consistency in maintaining coding standards across projects.

(iv) Predictive maintenance in software systems has become a strategic approach to ensuring system reliability. Machine learning models, trained on historical usage patterns, error logs, and performance metrics, can predict potential system failures or performance degradation [15]. This proactive approach ensures timely maintenance, minimizes downtime and improves system availability.

(v) AI algorithms have proven invaluable in optimizing CI/CD pipelines. By analyzing historical data, these algorithms identify bottlenecks, predict build outcomes, and recommend strategies for more efficient continuous integration and deployment processes [16]. This data-driven optimization contributes to faster and more reliable software delivery.

(vi) AI-driven tools providing personalized assistance to developers have become increasingly sophisticated. These tools understand individual coding styles, preferences, and common errors through machine learning models [17]. These tools enhance individual developers' productivity and coding proficiency by offering tailored suggestions and guidance during development.

## III. Methodology

This study utilizes a quasi-experimental design to compare AI-based code generation with manual coding, focusing on four key sorting algorithms: Merge sort, Bubble Sort, Insertion Sort, and Selection Sort. This deliberate selection of algorithms ensures a comprehensive assessment across diverse algorithmic paradigms, capturing their strengths and nuances. By involving MCA III semester students in software engineering lab work, the study tailors tasks to their existing knowledge base while ensuring ethical standards by briefing participants on the study's objectives and procedures. This approach aims to provide a robust and unbiased evaluation of both coding methods, enhancing understanding of their effectiveness and efficiency.

## IV. Analysis

Comparing and identifying the assessment between the two approaches is expected to demonstrate efficiency gains in terms of development time and lines of code. Still, they may have varying error rates compared to manually written code. A controlled experiment can be designed and conducted. Below is the proposed methodology for such an experiment:

Hypotheses:

- *Null Hypothesis (H0):* There is no significant difference in development time, lines of code, and error rates between generative AI-based and manually written code [20].
- *Alternative Hypothesis (H1):* Generative AI-based code will show efficiency gains in terms of development time, lines of code, and error rates compared to manually written code [21].

Group A (AI-generated code): Participants will use generative AI to generate code snippets for a specific task. Group B (Manual Coding): Participants will manually write code for the same task. To comprehensively compare AI-generated Sorting codes and manual code, a comprehensive analysis considering various metrics is necessary. Table I is the average time comparison between the AI-generated and manual coding.

(i). *Development Time:* The time to complete a coding task.
Development Time =Time Ended−Time Started

(ii*). Lines of Code:* The number of lines of code in a program.

(iii) *Error Rates:* The ratio of errors to the total number of code elements.
Error Rate=Number of Errors/LOC

TABLE I. COMPARES DEVELOPMENT TIME BETWEEN THE AI-GENERATED AND MANUAL CODING GROUPS

| Participant | Average Time for AI-Generated code | Average Time for Manual Coding |
|---|---|---|
| 1 | 8 minutes | 12 minutes |
| 2 | 10 minutes | 14 minutes |
| 3 | 9 minutes | 13 minutes |
| 4 | 11 minutes | 15 minutes |
| 5 | 7 minutes | 11 minutes |

Table I compares the average development time between the AI-generated and Manual Coding groups for the different algorithms. The values represent the time individual participants took in each group and the calculated mean development time for both groups.

*A. Development time:* From Table I, we have the following average times for AI-generated code and manual coding: AI-generated code group: [8, 10, 9, 11, 7] (mean = 9 minutes). Manual coding group: [12, 14, 13, 15, 11] (mean = 13 minutes) We can apply a paired t-test to evaluate whether a statistically significant difference exists between the mean development time for AI-generated and manual coding. The p-value for the test is 0.0, and the difference in development time was highly significant, indicating that AI-generated code allowed for faster task completion.

*B. Line of Code:* For the four sorting algorithms (Merge sort, Bubble Sort, Insertion Sort, and Selection Sort), we get the following means of lines of code: AI-generated code: [54.8, 48.6, 49.4, 47.8] (mean = 50.15 lines of code)Manual coding: [84.6, 78.8, 79.6, 78.2] (mean = 80.3 lines of code)A t-test can be applied to test whether the mean number of lines of code differs significantly between the two groups. The mean number of lines of code for AI-generated solutions (50.15 lines) was notably fewer than for manually coded solutions (80.3 lines). The p-value is significantly lower, confirming that the difference was statistically significant and highlighting that AI-generated code is more concise.

*C. Error Rates*: The error rates for both AI-generated and manually coded solutions (semantics/syntax) were examined, and differences in the occurrence of runtime errors were revealed. Statistical tests were applied to assess the significance of these differences. Tables II and III summarise the error rate for AI-generated and manual-generated codes.

TABLE II. THE ERROR RATE FOR THE AI-GENERATED CODE

| Algorithm | Errors (AI) | LOC (AI) | Error Rate (AI) |
|---|---|---|---|
| Merge sort | 3 | 54.8 | 0.0548 |
| Bubble Sort | 2 | 48.6 | 0.0412 |
| Insertion Sort | 3 | 49.4 | 0.0607 |
| Selection Sort | 1 | 47.8 | 0.0209 |

TABLE III. THE ERROR RATE FOR THE MANUAL CODE

| Algorithm | Errors (Manual) | LOC (Manual) | Error Rate (Manual) |
|---|---|---|---|
| Merge sort | 5 | 84.6 | 0.0591 |
| Bubble Sort | 4 | 78.8 | 0.0508 |
| Insertion Sort | 6 | 79.6 | 0.0754 |
| Selection Sort | 4 | 78.2 | 0.0512 |

We can apply a paired t-test to evaluate whether there is a statistically significant difference between the error rates for AI-generated and manual coding. The p-value for the test is 0.048, typically considered significant below 0.050. The paired t-tests showed significant efficiency gains in development time, lines of code and error rates for AI-generated code. Therefore, we reject the null hypothesis (H0) and accept the alternative hypothesis (H1), concluding that AI-generated code demonstrates clear efficiency advantages over manual coding.

## V. Result and Discussion

The results of our comparative analysis between AI-generated sorting codes and manually written codes across multiple algorithms align with the initial hypothesis. The hypothesis posited that generative AI-based code would demonstrate efficiency gains in terms of development time and lines of code while potentially exhibiting varying error rates compared to manually written code. The outcomes consistently support these expectations.

For calculating the development time across all sorting algorithms, the AI-generated group exhibited a statistically significant reduction in mean development time compared to the Manual Coding group. The efficiency gains ranged from 5 to 8 minutes, reflecting the potential time-saving benefits of AI-generated code. This outcome is consistent with the hypothesis that AI-based code generation can expedite coding.

Similarly, for lines of code, the AI-generated group consistently produced code snippets with significantly fewer lines than the Manual Coding group. The reduction in lines of code ranged from 28 to 30 lines on average, highlighting the code conciseness achieved through AI-based code generation. This aligns with the anticipated benefit of code brevity associated with AI-generated algorithms.

Meanwhile, the hypothesis proposed varying error rates for the error rates, and the observed results showed a nuanced picture. The AI-generated group generally exhibited lower error rates, fewer runtime errors, and logical issues across sorting algorithms. However, the difference in error rates varied among algorithms, suggesting the need for algorithm-specific considerations.

The results indicate that AI-generated sorting codes consistently offer efficiency advantages in terms of development time and lines of code. This suggests the potential for integrating AI-generated code in scenarios where rapid development and concise implementations are crucial. However, the varying error rates emphasize the importance of careful validation and testing, especially in critical applications.

## VI. Future Aspects

The successful implementation of generative AI in sorting algorithms unveils promising avenues for future exploration and refinement. Subsequent research endeavors could delve into algorithm-specific optimization, tailoring generative AI models to understand better and optimize specific sorting algorithms while addressing nuances in their implementation. Efforts in advancing training models aim to minimize error rates further, ensuring the reliability of AI-generated code. Scalability assessments will be crucial in evaluating how well AI-generated code can handle larger datasets and exploring its scalability in real-world scenarios.

Integrating AI-generated code seamlessly into the software development workflow is a prospective area of investigation, potentially establishing it as an integral tool for developers. Additionally, developing user-friendly interfaces will allow users to interact with and guide the AI in generating customized sorting algorithms, enhancing accessibility and usability. In conclusion, these results showcase the current potential of AI-generated code in revolutionizing sorting algorithms and laying the groundwork for future advancements and practical applications in software development.

## References


[1] N. Chitadze, "The Global Role of Artificial Intelligence in the Learning Process," in Digital Active Methodologies for Educative Learning Management, IGI Global, 2022, pp. 78-117.

[2] B. Dresp-Langley, O. K. Ekseth, J. Fesl, S. Gohshi, M. Kurz, and H. W. Sehring, "Occam's Razor for Big Data? On detecting quality in large unstructured datasets," Applied Sciences, vol. 9, no. 15, p. 3065, 2019.

[3] K. K. Ng, C. H. Chen, C. K. Lee, J. R. Jiao, and Z. X. Yang, "A systematic literature review on intelligent automation: Aligning concepts from theory, practice, and future perspectives," Advanced Engineering Informatics, vol. 47, p. 101246, 2021.

[4] A. K. Tyagi, T. F. Fernandez, S. Mishra, and S. Kumari, "Intelligent automation systems are at the core of Industry 4.0," in International Conference on Intelligent Systems Design and Applications, Cham: Springer International Publishing, 2020, pp. 1-18.

[5] S. Dey and S. W. Lee, "Multilayered review of safety approaches for machine learning-based systems in the days of AI," Journal of Systems and Software, vol. 176, p. 110941, 2021.

[6] L. Cao, K. Mohan, B. Ramesh, and S. Sarkar, "Adapting funding processes for agile IT projects: an empirical investigation," European Journal of Information Systems, vol. 22, no. 2, pp. 191-205, 2013.

[7] D. Batra, "Agile values or plan-driven aspects: which factor contributes more toward the success of data warehousing, business intelligence, and analytics project development?," Journal of Systems and Software, vol. 146, pp. 249-262, 2018.

[8] S. L. Wamba-Taguimdje, S. Fosso Wamba, J. R. Kala Kamdjoug, and C. E. Tchatchouang Wanko, "Influence of artificial intelligence (AI) on firm performance: the business value of AI-based transformation projects," Business Process Management Journal, vol. 26, no. 7, pp. 1893-1924, 2020.

[9] A. J. Scott, A world in emergence: cities and regions in the 21st century, Edward Elgar Publishing, 2012.

[10] M. K. Leung, A. Delong, B. Alipanahi, and B. J. Frey, "Machine learning in genomic medicine: a review of computational problems

[11] and data sets," Proceedings of the IEEE, vol. 104, no. 1, pp. 176-197, 2015.

[12] X. Hou et al., "Large language models for software engineering: A systematic literature review," arXiv preprint arXiv:2308.10620, 2023.

[13] J. Jacobs and L. Buechley, "Codeable objects: computational design and digital fabrication for novice programmers," in Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, 2013, pp. 1589-1598.

[14] T. Sharma et al., "A survey on machine learning techniques for source code analysis," arXiv preprint arXiv:2110.09610, 2021.

[15] Y. K. Dwivedi et al., "Artificial Intelligence (AI): Multidisciplinary perspectives on emerging challenges, opportunities, and agenda for research, practice and policy," International Journal of Information Management, vol. 57, p. 101994, 2021.

[16] B. Mohammed, I. Awan, H. Ugail, and M. Younas, "Failure prediction using machine learning in a virtualized HPC system and application," Cluster Computing, vol. 22, pp. 471-485, 2019.

[17] V. Grover, R. H. Chiang, T. P. Liang, and D. Zhang, "Creating strategic business value from big data analytics: A research framework," Journal of Management Information Systems, vol. 35, no. 2, pp. 388-423, 2018.

[18] A. S. George and A. H. George, "A review of ChatGPT AI's impact on several business sectors," Partners Universal International Innovation Journal, vol. 1, no. 1, pp. 9-23, 2023.

[19] S. K. Mishra, A. Gupta, A. Tomar and V. K. Gupta, "Health Care Prediction for Various Diseases using Computational Intelligence Approaches: A Review," 2023 World Conference on Communication & Computing (WCONF), RAIPUR, India, 2023, pp. 1-6.

[20] V. K. Gupta, A. Gupta, M. K. Singh, A. Gupta and A. Tomar, "Toxicity Detection Methodology of Adverse Outcome Pathways using Physicochemical Properties and Machine Learning Approaches," 2023 4th International Conference for Emerging Technology (INCET), Belgaum, India, 2023, pp. 1-7.

[21] M. K. Singh, P. Singh, V. K. Gupta, K. Mishra, and A. Gupta, "Performance Analysis of CNN Models with Data Augmentation in Rice Diseases," 2023 3rd Asian Conference on Innovation in Technology (ASIANCON), Ravet IN, India, 2023, pp. 1-5.